

Agentic AI

Naeemullah Khan

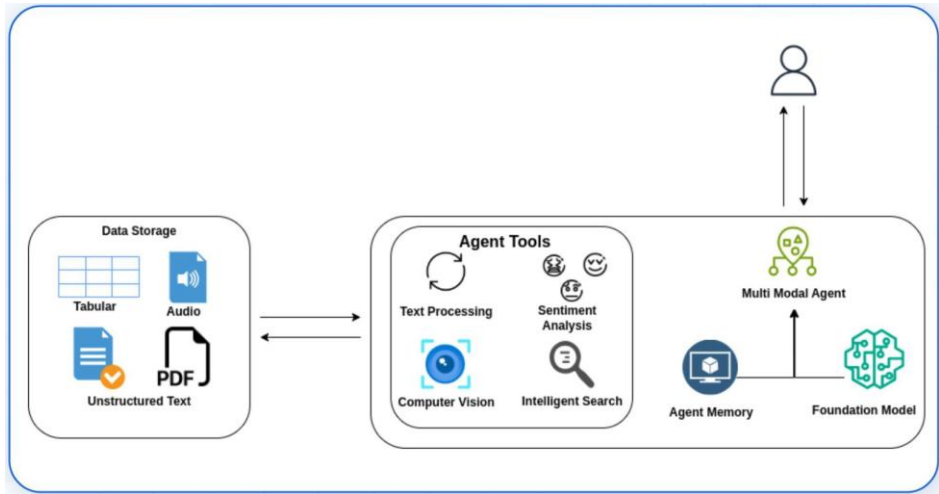
naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

August 18, 2026



Redrawn after Nath et al., “Generative AI and multi-modal agents in AWS: The key to unlocking new value in financial markets”, AWS Machine Learning Blog, 2023 (“Architecture Pattern: Multi-modal Agent”).

1. Motivation
2. Learning Outcomes
3. Agentic AI: Definitions, Reactive vs. Proactive Agents
4. Agent Loop, Tool Use, and Agentic RAG
5. Open-Source Agentic Frameworks
 1. A Taxonomy of Agent Frameworks
 2. Auto-GPT and BabyAGI
 3. Other Notable Frameworks

6. Code Generation

1. Benchmarks and Metrics
2. Methods: Context, Infilling, Retrieval, Execution Feedback
3. Representative Code LMs

7. Continual Learning in Agents

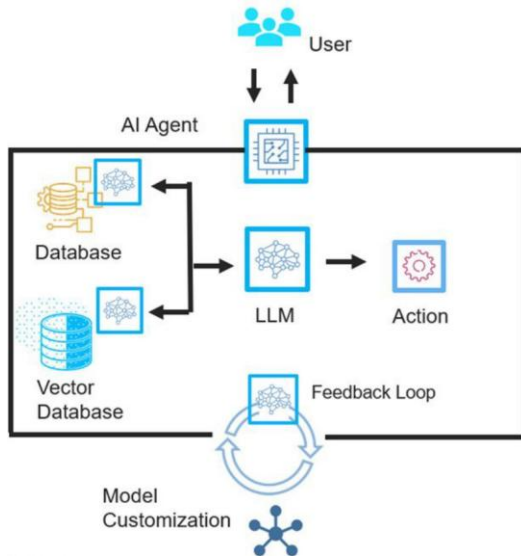
8. Safety, Ethics, and Control

9. Future Directions

- ▶ A language model answers; an **agent** acts: it pursues goals over multiple steps, using tools, memory, and feedback from its environment.
- ▶ Modern AI agents must perceive, reason, plan, and act in complex environments — often across multiple modalities, building on the vision-language models of the previous lecture.
- ▶ **Agentic AI** aims to build goal-directed, autonomous, and continuously learning systems on top of LLMs.
- ▶ The agent's most general interface to the world is **executable code** — so this lecture also covers how models write code, and how that ability is measured.
- ▶ The first agent frameworks (Auto-GPT, BabyAGI) opened new research frontiers in decision-making, robotics, and assistive AI — and new failure modes, which is why safety and control are part of the core material.

- ▶ Understand the principles of agentic AI and agentic loops.
- ▶ Describe how an LLM acts through function calling, and what MCP standardises.
- ▶ Differentiate between reactive and proactive agent behavior.
- ▶ Explain how code generation is evaluated (HumanEval, pass@k, SWE-bench, match-based metrics) and the methods behind modern code LMs (context engineering, infilling, retrieval, execution feedback).
- ▶ Describe continual learning challenges in agentic systems.
- ▶ Critically evaluate frameworks like Auto-GPT and BabyAGI.
- ▶ Reflect on safety, control, and ethical issues in agentic AI.

Agentic AI



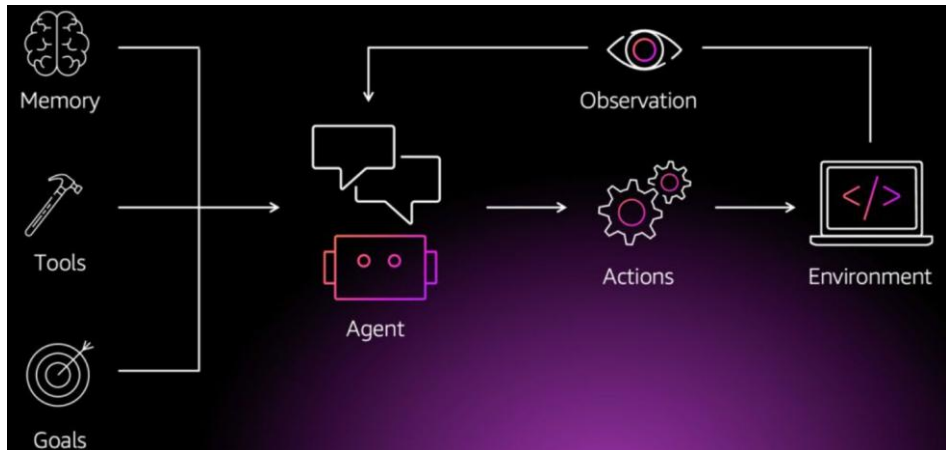
Source: Cadence, "What is Agentic AI?", https://www.cadence.com/en_US/home/explore/agentic-ai.html.

- ▷ AI systems that **perceive**, **reason**, **plan**, and **act** to fulfill goals.
- ▷ **Agent loop**: Observe → Plan → Act → Reflect → Learn
- ▷ Inspired by cognitive science, robotics, and autonomous agents.

Core Components:

- ▷ Memory
- ▷ Planning
- ▷ Tool-use (plugins, APIs)
- ▷ Feedback-driven behavior

What is Agentic AI?

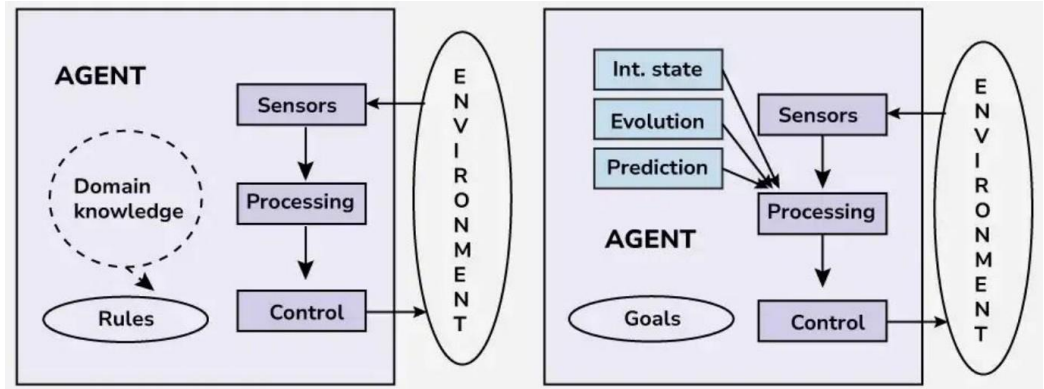


Source: Kipkemboi, Moura, Singh & Tsakpinis, "Build agentic systems with CrewAI and Amazon Bedrock", AWS Machine Learning Blog, 2025.

Feature	Reactive	Proactive
Response	Immediate	Goal-oriented
Planning	None	Yes
Learning	Event-triggered	Self-initiated
Example	Chatbots	Auto-GPT, Personal Assistants

Proactive agents generate goals, schedule actions, and evaluate results even without user prompts.

Reactive vs. Deliberative Agents



A reactive agent maps percepts to actions through fixed rules. A deliberative agent keeps internal state, predicts how the environment will evolve, and selects actions against a goal. Source: GeeksforGeeks, "Reactive vs Deliberative AI Agents".

- ▷ **Perceive:** Multimodal input (text, vision, memory)
- ▷ **Interpret:** NLP/vision models
- ▷ **Plan:** Task decomposition (e.g., LangChain chains)
- ▷ **Act:** API/tool calling
- ▷ **Reflect & Learn:** Refine memory or adjust goals

Applications:

- ▷ Personal AI assistants
- ▷ Research co-pilots
- ▷ Automated workflows

- ▷ **Function calling** is how an LLM acts: each tool is described to the model as a function with a name, a description, and a JSON schema for its parameters.
- ▷ The model does not execute anything. It emits a structured call (function name + arguments); the runtime executes it and feeds the result back as a new message.
- ▷ **The loop:** model receives the task → emits a call → runtime returns the result → model reasons over it → another call or a final answer. This is the ReAct pattern covered earlier, with the action step now carried by the API instead of by parsed text.
- ▷ Modern APIs support **parallel calls** (several independent tools in one step) and **structured outputs** (forcing the reply itself into a schema).

Tool definition given to the model:

```
{"name": "get_weather",  
  "description": "Current weather for a city",  
  "parameters": {"type": "object",  
    "properties": {"city": {"type": "string"}},  
    "required": ["city"]}}
```

User: "Do I need an umbrella in Jeddah today?"

Model -> call: get_weather(city="Jeddah")

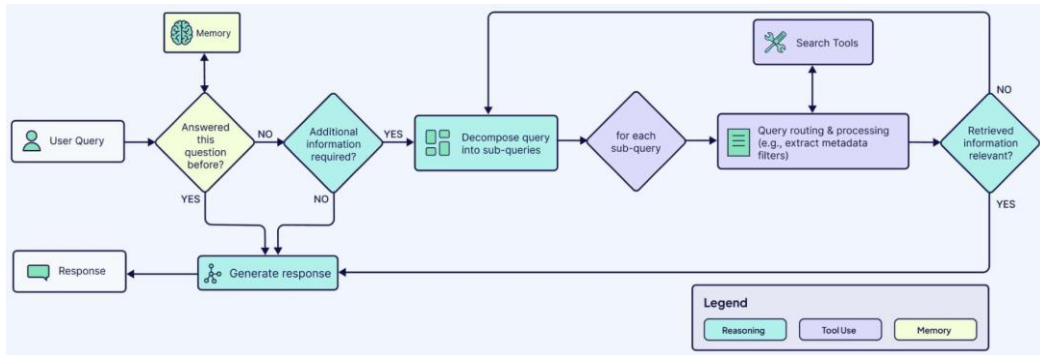
Runtime -> result: {"condition": "rain", "temp_c": 24}

Model -> "Yes, take one: it is raining in Jeddah (24 C)."

- ▷ **The problem.** Function calling describes tools *to one model, in one vendor's format*. Every new model or agent framework means rewriting the same integrations, so M agents times N tools costs $M \times N$ adapters.
- ▷ **Model Context Protocol (MCP)**, released by Anthropic in November 2024, is an open protocol for that connection, modelled on the Language Server Protocol used by code editors. It reduces the cost to $M + N$.
- ▷ **Three roles:** a **host** is the application the user talks to; a **client** lives inside the host and holds one connection; a **server** wraps a data source or service and advertises what it offers.
- ▷ A server exposes three kinds of thing: **tools** (actions the model can invoke), **resources** (data it can read), and **prompts** (templates the user can select). Messages travel as JSON-RPC.
- ▷ The payoff is that a filesystem, database, or issue-tracker server written once works with any MCP-capable agent, whoever built it.

Specification: <https://modelcontextprotocol.io/specification/latest>

- ▷ **Fixed RAG pipeline:** retrieve once for the raw query, then generate. Fails when the query is vague, multi-part, or needs information the first retrieval missed.
- ▷ **Agentic RAG** makes retrieval a tool the model controls:
 - **Decide when:** answer directly from parametric knowledge, or retrieve.
 - **Rewrite the query:** decompose multi-part questions into targeted searches.
 - **Iterate:** retrieve, read, notice what is missing, retrieve again.
 - **Judge the evidence:** discard irrelevant passages before answering.
- ▷ Costs more latency and tokens per query; pays off on multi-hop questions and heterogeneous document collections.



Source: Newhauser, Yadav, Monigatti & Çelik, "What Are Agentic Workflows? Patterns, Memory, Use Cases, and Examples", Weaviate blog, 2025.

Surveying (multimodal) LLM agents, Xie et al. sort every framework along two axes — **who plans** and **what is remembered**:

	Planner	Long-term memory
Type I	closed-source LLM, steered by prompts	none
Type II	open LLM, fine-tuned on action data	none
Type III	either	indirect : retrieved from an external store
Type IV	either	native : built into the planner itself

- ▷ Indirect memory is the Agentic RAG pattern from the previous section: experience lands in a vector store and returns through retrieval — Auto-GPT and BabyAGI (next) both live here.
- ▷ Multi-agent frameworks add a third axis: specialised agents dividing the work, with or without a *shared* long-term memory.

Xie et al., “Large Multimodal Agents: A Survey,” [arXiv:2402.15116](https://arxiv.org/abs/2402.15116).

Open-Source Agentic Frameworks

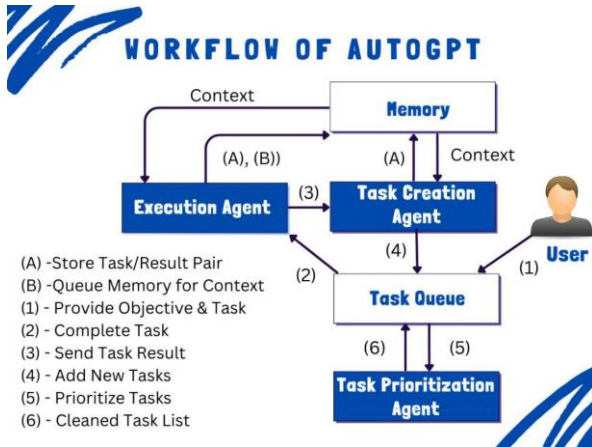
- ▶ **Open-source project** (Toran Bruce Richards, March 2023) that chained GPT-4 calls into a self-directed agent.
- ▶ Used **long-term memory** in a vector store, a file workspace, and self-generated tasks.
- ▶ **Requirements:**
 - Task input
 - Internet / API access
 - Feedback loop
- ▶ **Status:** the original agent now ships as `classic/` and is unsupported; the project has become a visual builder, the AutoGPT Platform.

GitHub: <https://github.com/Significant-Gravitas/AutoGPT>

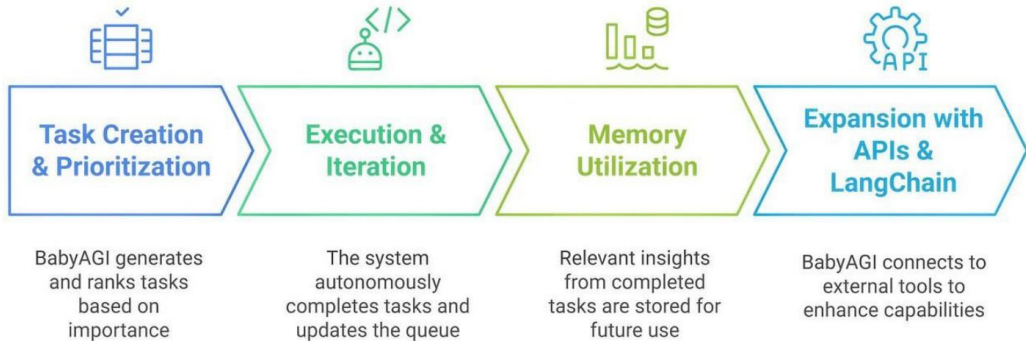
- ▷ **Lightweight Python agent loop** (Yohei Nakajima, March 2023), around 140 lines
- ▷ Generates tasks from a high-level objective
- ▷ Runs tasks, stores results in a vector store, and generates new tasks iteratively
- ▷ Emphasizes simplicity and minimalism

Original code: https://github.com/yoheinakajima/babyagi_archive

The babyagi repository itself now hosts a different, self-building agent framework.



Three LLM calls with one shared task queue and one memory store. Architecture after Y. Nakajima, “Task-Driven Autonomous Agent” (2023); this rendition is labelled “Workflow of AutoGPT” by its publisher, but the three named agents are BabyAGI’s.



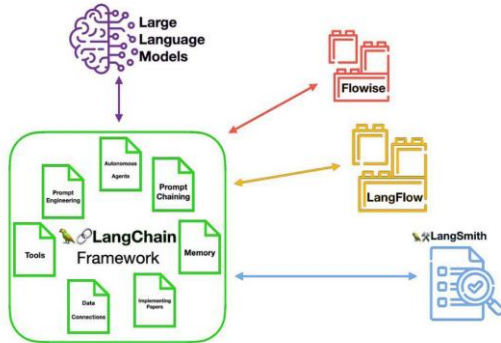
- ▶ **The measured results were poor.** GAIA asks 466 questions that are conceptually simple for a person but need tool use to answer. Human respondents score 92%; GPT-4 with plugins scores 15%. Auto-GPT on a GPT-4 backend manages 14.4% on the easiest tier and 0.4% on the middle tier, behind plain GPT-4 with hand-picked plugins.
- ▶ **Where the loops broke:** no reliable stopping criterion, so agents revisited the same subtask; errors compounded because nothing checked a step before the next built on it; and cost grew with every retry.
- ▶ AgentBench, across eight interactive environments, traces the gap to long-term reasoning, decision-making, and instruction-following rather than to the scaffolding around the model.
- ▶ **What survived:** the loop and its parts, a task queue, a memory store, tools behind a schema. What did not is the assumption that full autonomy from a one-line objective is the right target. Current systems narrow the task, verify each step, and keep a person in the loop for consequential actions.

Sources: Mialon et al., “GAIA”, [arXiv:2311.12983](https://arxiv.org/abs/2311.12983), Table 4; Liu et al., “AgentBench”, ICLR 2024, [arXiv:2308.03688](https://arxiv.org/abs/2308.03688).

- ▶ **LangChain:** Framework for chaining LLMs with tools, APIs, and memory; since version 1.0 (2025) its headline abstraction is agent construction on the LangGraph runtime.
- ▶ **CrewAI:** Python framework and hosted platform for orchestrating role-playing agents that collaborate on a task.
- ▶ **SuperAGI:** Dev-first open-source autonomous-agent framework with toolkits, agent memory, and telemetry. Largely dormant: last release January 2024.
- ▶ **OpenHands** (renamed from OpenDevin in August 2024): open platform for software-development agents, giving the agent a sandboxed shell, editor, and browser behind an event-stream architecture.

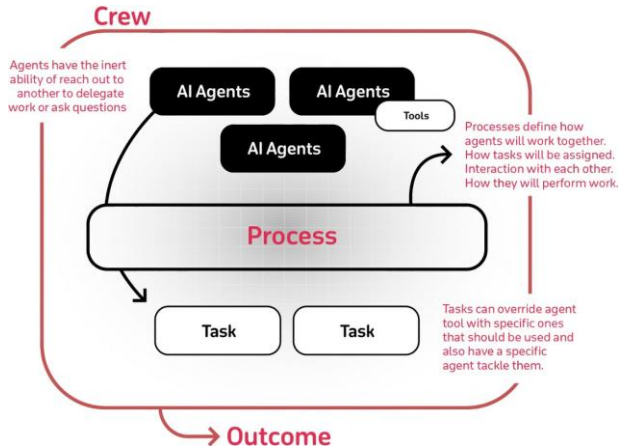


LangChain Ecosystem

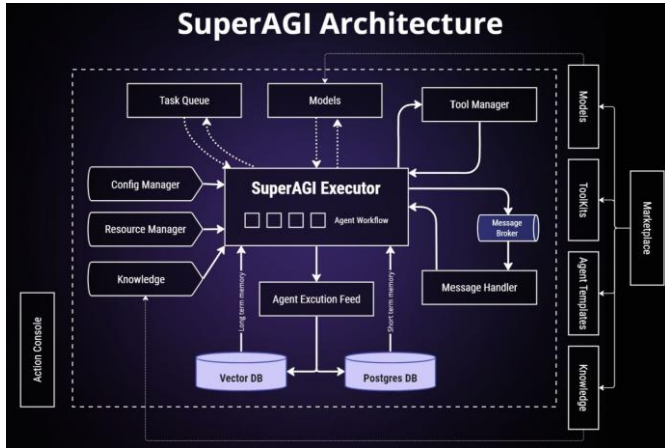


www.cobusgreyling.com

Source: C. Greyling, "The Growing LangChain Ecosystem".



A crew is a set of role-playing agents; a process defines how they hand work between each other. Source: CrewAI documentation, <https://docs.crewai.com>.

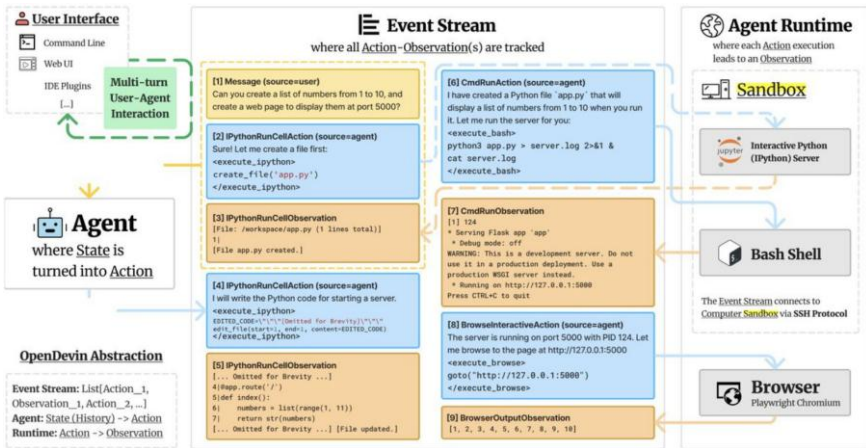


Source: SuperAGI project, <https://github.com/TransformerOptimus/SuperAGI>.

SuperAGI Core Features & Capabilities

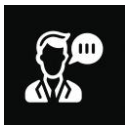


Source: Servifyspheresolutions, "AutoGPT vs. BabyAGI vs. SuperAGI: Which AI Agent is Smarter for Autonomous Tasks?", Generative AI (Medium), 2025.



Every action and observation is appended to one event stream, which doubles as the agent's history and its memory. Source: Wang et al., "OpenHands: An Open Platform for AI Software Developers as Generalist Agents", ICLR 2025, [arXiv:2407.16741](https://arxiv.org/abs/2407.16741).

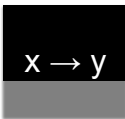
- Generate executable code as an interface to a program
- Why do it?
 - Accelerates software engineering
 - Enables models to access tools (more in a later class)



Natural language



Code context



Inputs and outputs



Screenshots

Code

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f"Name: {self.name}, Age: {self.age}")

    def years_until_100(self):
        years_left = 100 - self.age
        if years_left <= 0:
            print(f"{self.name} is already 100 years old or more!")
        else:
            print(f"{self.name} has {years_left} years until they turn 100.")
```

- A code completion engine within your IDE (VSCode)

```
# load hello.csv
data = np.loadtxt('hello.csv', delimiter=',', skiprows=1)

# add together the two largest numbers in the first column
result = np.sum(np.sort(data[:, 0])[-2:])

💡 print how much bigger these two numbers are than the average of the first column
result = result - np.mean(data[:, 0])
```

<https://youtu.be/xe3ol0rTeaQ>



6 Create a React app that looks like this screenshot.

To create a React app that looks similar to the screenshot, you can follow these steps:

1. Set up a new React project using Create React App or your preferred method.
2. Create a component for the chat interface, such as `ChatInterface`.
3. Inside the `ChatInterface` component, add a header with the app name "Claude"

Here's a simplified example of the `ChatInterface` component:

```
jsx
import React from 'react';
import styled from 'styled-components';

const ChatContainer = styled.div`
  /* Add styles for the chat container */
`;

const Header = styled.header`
  /* Add styles for the header */
`;

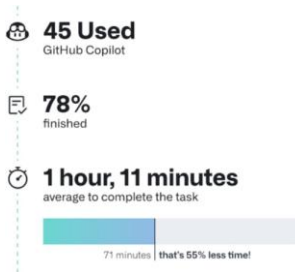
const ChatSection = styled.section`
  /* Add styles for the chat section */
`;

const CapabilityCard = styled.div`
  /* Add styles for the capability cards */
`;

const ChatInterface = () => {
  return (
    <ChatContainer>
      <Header>
        <ChatSection>
          <CapabilityCard>
            <CapabilityCard>
              <CapabilityCard>
            </CapabilityCard>
          </ChatSection>
        </Header>
      </ChatContainer>
    )
  );
}
```

Why do we do it?

- Making software is important!
 - US software developers make \$175B per year
- Code generation leads to large improvements in productivity (Github 2023)



- Code has strict grammar
- We know the code semantic flow
- Code is executable
- Code is created more incrementally

Code Generation - Subtasks and Datasets

- Examples of usage of the Python standard library
- 164 test examples
- Includes docstring, some example inputs/outputs, and tests

```
def solution(lst):  
    """Given a non-empty list of integers, return the sum of all of the odd elements  
    that are in even positions.  
  
    Examples  
    solution([5, 8, 7, 1]) ==>12  
    solution([3, 3, 3, 3, 3]) ==>9  
    solution([30, 13, 24, 321]) ==>0  
    """  
    return sum(lst[i] for i in range(0, len(lst)) if i % 2 == 0 and lst[i] % 2 == 1)
```

```
def encode_cyclic(s: str):  
    """  
    returns encoded string by cycling groups of three characters.  
    """  
    # split string to groups. Each of length 3.  
    groups = [s[(3 * i):min((3 * i + 3), len(s))] for i in range((len(s) + 2) // 3)]  
    # cycle elements in each group. Unless group has fewer elements than 3.  
    groups = [(group[1:] + group[0]) if len(group) == 3 else group for group in groups]  
    return "".join(groups)  
  
def decode_cyclic(s: str):  
    """  
    takes as input string encoded with encode_cyclic function. Returns decoded string.  
    """  
    # split string to groups. Each of length 3.  
    groups = [s[(3 * i):min((3 * i + 3), len(s))] for i in range((len(s) + 2) // 3)]  
    # cycle elements in each group.  
    groups = [(group[-1] + group[:-1]) if len(group) == 3 else group for group in groups]  
    return "".join(groups)
```

Metric: Pass@K (Chen et al. 2021)

- Basic idea: “if we generate K examples, will at least one of them pass unit tests”
- Generating only K will result in high variance, so we generate $N > K$ with C correct answers, and then calculate expected value

$$\text{pass}@k := \mathbb{E}_{\text{Problems}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right]$$

Broader Domains: CoNaLa/ODEX

(Yin et al. 2018, Wang et al. 2022)

- CoNaLa: Broader data scraped from StackOverflow
- ODEX: Adds execution-based evaluation
- Wider variety of libraries

Removing duplicates in lists ← Intent

Question

406 Pretty much I need to write a program to check if a list has any duplicates and if it does it removes them and returns a new list with the items that weren't duplicated/removed. This is what I have but to be honest I do not know what to do.

```
def remove_duplicates():  
    t = ['a', 'b', 'c', 'd']  
    t2 = ['a', 'c', 'd']  
    for t in t2:  
        t.append(t.remove())  
    return t
```

780 The common approach to get a unique collection of items is to use a `.set`. Sets are unordered collections of distinct objects. To create a set from any iterable, you can simply pass it to the built-in `set()` function. If you later need a real list again, you can similarly pass the set to the `.list()` function.

The following example should cover whatever you are trying to do:

```
>>> t = [1, 2, 3, 1, 2, 5, 6, 7, 8]  
[1, 2, 3, 1, 2, 5, 6, 7, 8]  
>>> list(set(t))  
[1, 2, 3, 5, 6, 7, 8]  
>>> s = [1, 2, 3]  
>>> list(set(t) - set(s))  
[8, 5, 6, 7]
```

Context 1
Snippet 1

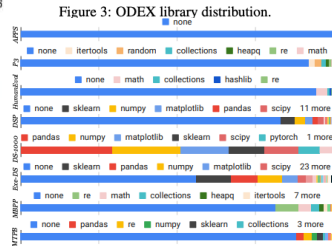
As you can see from the example result, the original order is not maintained. As mentioned above, sets themselves are unordered collections, so the order is lost. When converting a set back to a list, an arbitrary order is created.

222 FWIW, the new (v2.7) Python way for removing duplicates from an iterable while keeping it in the original order is:

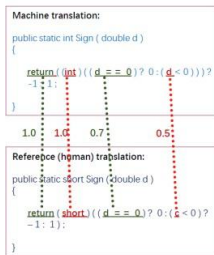
```
>>> from collections import OrderedDict  
>>> list(OrderedDict.fromkeys('abracadabra'))  
['a', 'b', 'r', 'c', 'd', 'a']
```

Context 2
Snippet 2

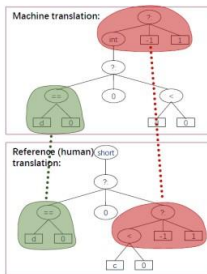
Answers



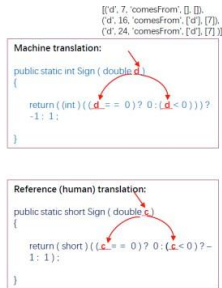
- Issues w/ execution-based evaluation:
 - Requires that code be easily executable (requires unit tests and hard in large libraries)
 - Ignores stylistic considerations
- **BLEU**: consider text n-gram overlap with human code
- **CodeBLEU**: also considers syntax and semantic flow (Ren et al. 2020)



Weighted N-Gram Match



Syntactic AST Match



Semantic Data-flow Match

- It is also possible to use embedding-based methods to compare code
- **CodeBERTScore**: BERTScore with CodeBERT trained on lots of code (Zhou et al. 2023)
- Leads to better correlation w/ human judgement and execution accuracy

Metric	Java		C++		Python		JavaScript	
	τ	r_s	τ	r_s	τ	r_s	τ	r_s
BLEU	.481	.361	.112	.301	.393	.352	.248	.343
CodeBLEU	.496	.324	.175	.201	.366	.326	.261	.299
ROUGE-1	.516	.318	.262	.260	.368	.334	.279	.280
ROUGE-2	.525	.315	.270	.273	.365	.322	.261	.292
ROUGE-L	.508	.344	.258	.288	.338	.350	.271	.293
METEOR	.558	.383	.301	.321	.418	.402	.324	.415
chrF	.532	.319	.319	.321	.394	.379	.302	.374
CrystalBLEU	.471	.273	.046	.095	.391	.309	.118	.059
CodeBERTScore	.553	.369	.327	.393	.422	.415	.319	.402

Table 1: Kendall-Tau (τ) and Spearman (r_s) correlations of each metric with the functional correctness on HumanEval in multiple languages. The correlation coefficients are reported as the average across three runs. Standard deviation is provided in Table 3.

Metric	τ	r_p	r_s
BLEU	.374	.604	.543
CodeBLEU	.350	.539	.495
ROUGE-1	.397	.604	.570
ROUGE-2	.429	.629	.588
ROUGE-L	.420	.619	.574
METEOR	.366	.581	.540
chrF	.470	.635	.623
CrystalBLEU	.411	.598	.576
CodeBertScore	.517	.674	.662

Table 2: The Kendall-Tau (τ), Pearson (r_p) and Spearman (r_s) correlation with human preference. The best performance is **bold**. The correlation coefficients are reported as the average across three runs. Standard deviations are provided in Table 4.

Data Science Notebooks: ARCADE

(Yin et al. 2022)

- Data science notebooks (e.g. Jupyter) allow for incremental implementation
- Allows evaluation of code in context

```
[1] import pandas as pd
C1 df = pd.read_csv('dataset/Gamepass_Games_v1.csv')

[2] U1 Extract min and max hours as two columns
def get_avg(x):
    try: return float(x[0]), float(x[1])
    except: return 0, 0
df['min'], df['max'] = zip(*df['TIME'].str.replace(
    ' hours', '').str.split('-').apply(get_avg))
C2

[3] df['ADDED'] = pd.to_datetime(
C3 df['ADDED'], format='%d %b %y', errors='coerce')

[4] U2 In which year was the most played game added?
df['GAMERS'] = df['GAMERS'].str.replace(
    ' ', '').astype(int)
C4 added_year = df[df['GAMERS'].idxmax()]['ADDED'].year

[5] U3 For each month in that year, how many games that
has a rating of more than four?
df[(df['ADDED'].dt.year == added_year) &
(df['RATING'] > 4)].groupby(
C5 df['ADDED'].dt.month)['GAME'].count()

[6] U4 What is the average maximum completion time for
all fallout games added in 2021?
fallout = df[df['GAME'].str.contains('Fallout')]
fallout.groupby(fallout['ADDED'].dt.year).get_group(
C6 2021)['max'].mean()

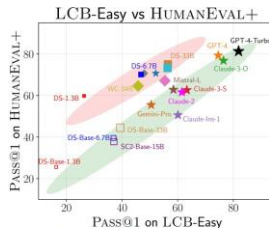
[7] U5 What is the amount of games added in each year
for each month? (show a table with index as years,
columns as months and fill null values with 0)
pd.pivot_table(df, index=df['ADDED'].dt.year, ...,
aggfunc=np.count_nonzero,
fill_value='0').rename_axis(
C7 index='Year', columns='Month')
```

Figure 1: An example of a computational notebook adapted from our dataset, with examples of reading and preprocessing data (cell c_1), data wrangling (cell c_2, c_3), and data analysis (cells $c_3 - c_7$). Annotated NL intents are shown in green.

An Aside: Dataset Leakage

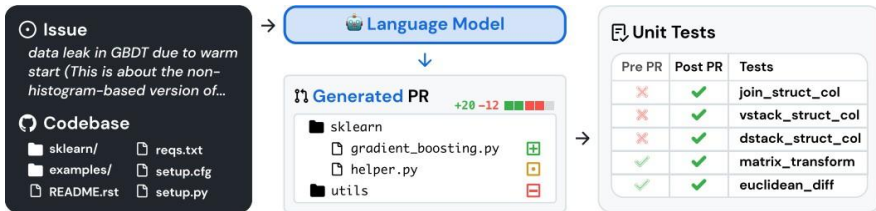
- Leakage of datasets is a big problem
- ARCADE shows that novel notebooks are harder than online notebooks
- LiveCodeBench (Jain et al. 2023) shows that some code LMs outperform on HumanEval

$pass@k$	Existing 5	New 5
INCODER 1B	30.1	3.8
INCODER 6B	41.3	7.0
CODEGEN _{multi} 350M	13.3	1.0
CODEGEN _{multi} 2B	25.0	2.7
CODEGEN _{multi} 6B	28.0	3.0
CODEGEN _{multi} 16B	31.2	4.6
CODEGEN _{mono} 350M	18.9	1.9
CODEGEN _{mono} 2B	35.8	6.5
CODEGEN _{mono} 6B	42.1	8.9
CODEGEN _{mono} 16B	46.7	12.0
PALM 62B (1.3T Tokens)	49.7	12.5
+ Python Code	58.8 +9.1	21.4 +8.9
+ Notebooks (PACHINCo)	64.6 +7.8	30.6 +9.2
- Schema Description	60.5 -4.1	22.7 -7.9



(Jiminez et al. 2023)

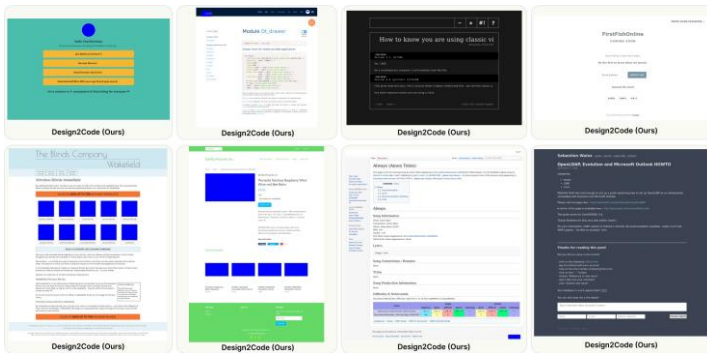
- Issues from GitHub + codebases -> pull request



- Requires long-context understanding, precise implementation

(Si et al. 2024)

- Code generation from web sites



- Also proposed Design2Code model

Code Generation - Methods

- Feed the previous code to an LM
- Virtually all serious LMs are trained on code nowadays, and have the ability to complete queries
- More on specific LMs later!
- Note: temperature settings are important, set to lower values

(Fried et al. 2022)

- In code generation, we often want to fill in code
- Solution: train for infilling

Training

Original Document

```
def count_words(filename: str) -> Dict[str, int]:  
    """Count the number of occurrences of each word in the file."""  
    with open(filename, 'r') as f:  
        word_counts = {}  
        for line in f:  
            for word in line.split():  
                if word in word_counts:  
                    word_counts[word] += 1  
                else:  
                    word_counts[word] = 1  
    return word_counts
```

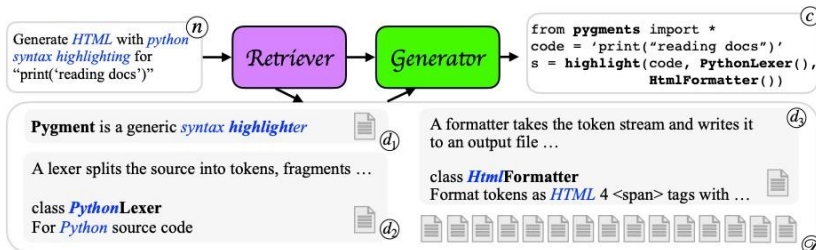
Masked Document

```
def count_words(filename: str) -> Dict[str, int]:  
    """Count the number of occurrences of each word in the file."""  
    with open(filename, 'r') as f:  
        <MASK:0> in word_counts:  
            word_counts[word] += 1  
        else:  
            word_counts[word] = 1  
    return word_counts  
<MASK:0> word_counts = {}  
    for line in f:  
        for word in line.split():  
            if word <EOM>
```

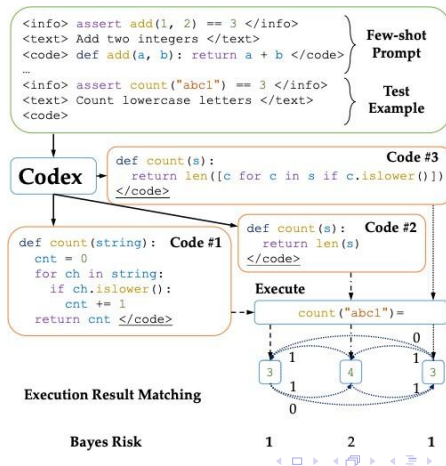
- Current code context
- Description of issue to fix
- Repo context
- Open tabs

- **Extract prompt** given current doc and cursor position
- Identify **relative path and language**
- Find **most recently accessed** 20 files of the same language
- **Include:** text before, text after, similar files, imported files, metadata about language and path
- **TL;DR:** lots of prompt engineering to get most useful context in the prompt

- Retrieve similar code from online, and fill it in with a retrieval-augmented LM (Hayati et al. 2018)
- Particularly, in code there is also documentation, which can be retrieved (Zhou et al. 2022)

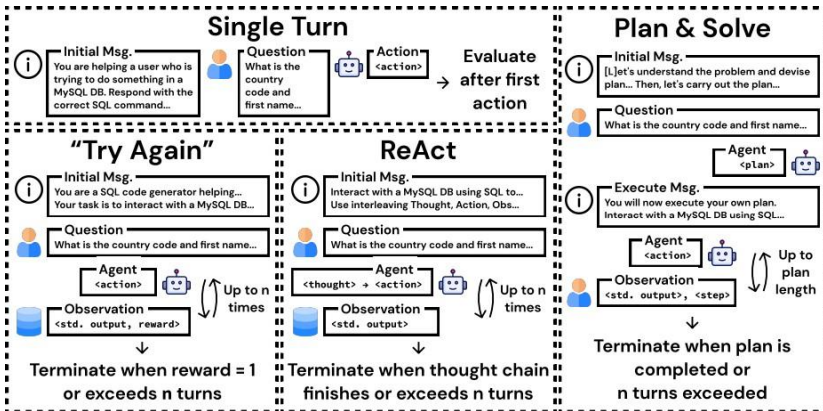


- Code can be executed, so we can use this to check results!



Fixing Based on Error Messages

- e.g. InterCode (Yang et al. 2023)



Code Generation - Representative

Code LMs

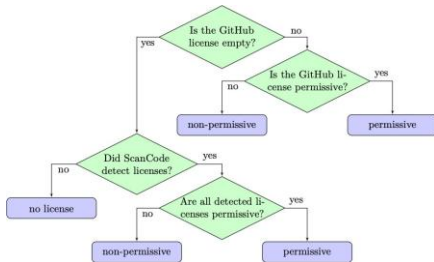
(Chen et al. 2022)

- **Creator:** OpenAI
- **Model:** Originally continued training from GPT-3
- **Data:** lots of data from GitHub
- Powers (powered?) GitHub Copilot

- **Creator:** Big Science (Hugging Face + Service Now)
- **Architecture:** Mostly LLaMa-style, w/ 3B, 7B, 15B variants, reconfigure RoPE for longer context
- **Data:** Trained on code (the stack), GitHub issues, pull requests, jupyter notebooks, Kaggle notebooks, documentation, intermediate representations like LLVM, NL datasets
- **Preproc:** Add metadata tags (e.g. repo name and file name) 50% of the time (to allow it to be used at test). Allow for infilling
- **Training:** Train for 4-5 epochs, 3T+ tokens total

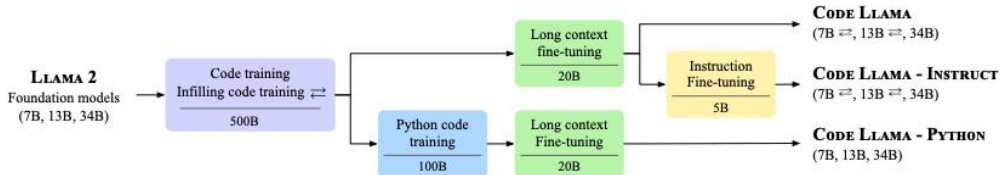
The Stack 2

- Code pre-training dataset w/ license considerations



Language	The-stack-v1-dedup		The-stack-v2-dedup		The-stack-v2-swh-full	
	Size (GB)	Files (M)	Size (GB)	Files (M)	Size (GB)	Files (M)
Assembly	1.58	0.25	13.02	0.77	7.74	0.70
Batchfile	0.29	0.25	2.11	1.13	1.02	0.99
C	57.43	8.53	202.05	20.78	114.92	19.18
C#	46.29	10.84	239.89	51.23	169.75	48.49
C++	50.89	6.37	353.89	43.18	211.33	42.23
CMake	0.45	0.19	2.58	1.74	2.27	1.70
CSS	22.61	2.99	161.68	23.87	8.00	1.88
Dockerfile	0.572	0.42	1.27	1.90	1.21	1.88
Fortran	0.17	1.84	4.66	0.27	3.61	0.26
Go	25.74	4.73	54.60	9.30	25.83	8.62
Haskell	2.36	0.54	5.11	1.25	4.17	1.23
HTML	146.76	9.53	2,419.87	90.23	99.09	5.23
Java	89.30	20.15	548.00	154.28	199.68	62.27
JavaScript	141.65	21.11	1,115.42	108.87	199.99	66.91
Julia	1.54	0.30	6.12	0.45	1.83	0.43
Lua	3.28	0.56	33.91	2.35	15.22	2.24
Makefile	1.49	0.66	21.30	4.22	5.19	2.78
Markdown	75.25	21.0	281.04	82.78	244.17	81.42
Perl	2.63	0.39	7.82	1.15	5.66	1.06
PHP	66.84	15.90	224.59	46.03	183.70	45.14
PowerShell	1.25	0.27	3.97	0.68	2.46	0.66
Python	64.30	12.96	233.29	56.93	191.61	56.19
R	0.30	0.04	22.39	5.15	19.05	4.29
Ruby	7.14	3.41	31.70	17.79	23.38	17.51
Rust	9.53	1.38	15.60	2.22	12.43	2.19
Scala	4.86	1.36	12.73	4.45	11.30	4.32
Shell	3.38	22.69	19.82	10.68	13.51	10.01
SQL	12.22	0.99	281.45	5.29	35.75	4.52
Swift	0	0	23.76	7.23	22.32	7.16
TeX	5.44	0.55	35.86	3.19	30.01	2.86
TypeScript	28.82	10.64	61.01	23.85	49.14	23.28
Visual Basic	1.49	0.16	16.63	1.06	7.48	0.81
Total	875.85	181.00	6,457.14	784.30	1,922.82	528.44

- **Creator:** Meta
- **Architecture:** Same as LLaMa 2 (7B, 13B, 34B and 70B), but trained on longer input contexts (100k) with longer RoPE
- **Data:** Trained on deduped code + synthetically created instruction data
Instruction data created by prompting LLaMa2 for coding problems
- **Training:** Incremental w/ various datasets



(Guo et al. 2024)

- **Data:** 87% source, 10% English from Markdown and StackExchange, 3% Chinese
- **Preproc:** Standard preproc, but also include library dependencies
- **Architecture:** LLaMa-like, 1.3B, 6.7B, and 33B, including reconfigured RoPE
- **Training:** Overall 2T tokens

Which to Use?

- All have somewhat similar performance, and are compared in StarCoder paper
- DeepSeekCoder seems to be strong on standard programming tasks
- StarCoder seems to be strong on data science notebooks

Future Directions

- ▶ **Multimodal grounding in real-world environments:** How can agents connect language, vision, and action in dynamic, noisy settings? SayCan is one line of attack: score what the language model *wants* to do against what the robot can actually do here.
- ▶ **World model integration:** Give the agent a learned internal model of how its environment evolves, so it can plan against predicted states rather than only react to observed ones.
- ▶ **Agent collaboration:** Enabling effective communication and coordination in multi-agent systems.
- ▶ **Scaling memory-efficient agents:** Designing architectures that scale to long-term, large-scale memory without prohibitive costs.
- ▶ **Robust reward models:** Developing reliable reward and feedback mechanisms for open-ended, real-world tasks.
- ▶ **Meta-cognition:** Building agents that can introspect, self-monitor, and adapt their own reasoning processes.

- ▶ **Agentic AI** adds goals, memory, and tools to a language model. **Function calling** is the mechanism; **MCP** standardises the interface.
- ▶ **The first autonomous frameworks (Auto-GPT, BabyAGI)** established the loop but performed poorly on real tasks. Their components survived; the assumption of full autonomy did not.
- ▶ **Code generation** is the agent's strongest actuator: executability gives cheap, trustworthy feedback — unit-test evaluation (pass@k, SWE-bench), execution-driven repair, and verifiable rewards all flow from it.
- ▶ **Continual learning** remains open: agents must absorb new tasks without overwriting old ones.
- ▶ Agency turns a wrong answer into a wrong action, which is why safety and oversight get their own treatment next.

- 1 Yao, S., Zhao, J., Yu, D., et al. (2023).
ReAct: Synergizing Reasoning and Acting in Language Models.
ICLR 2023. [arXiv:2210.03629](https://arxiv.org/abs/2210.03629)
- 2 Anthropic (2024).
Model Context Protocol: specification and SDKs.
<https://modelcontextprotocol.io/specification/latest>
- 3 Richards, T. B. (Torantulino) et al. (2023).
AutoGPT.
<https://github.com/Significant-Gravitas/AutoGPT>
- 4 Nakajima, Y. (2023).
BabyAGI (original task-driven agent; archived 2024).
https://github.com/yoheinakajima/babyagi_archive
- 5 Wang, X., Li, B., Song, Y., et al. (2025).
OpenHands: An Open Platform for AI Software Developers as Generalist Agents.
ICLR 2025. [arXiv:2407.16741](https://arxiv.org/abs/2407.16741)

- 6 Mialon, G., Fourrier, C., Swift, C., Wolf, T., LeCun, Y., Scialom, T. (2023).
GAIA: a Benchmark for General AI Assistants.
[arXiv:2311.12983](https://arxiv.org/abs/2311.12983)
- 7 Liu, X., Yu, H., Zhang, H., et al. (2024).
AgentBench: Evaluating LLMs as Agents.
ICLR 2024. [arXiv:2308.03688](https://arxiv.org/abs/2308.03688)
- 8 Ahn, M., Brohan, A., Brown, N., Chebotar, Y., et al. (2022).
Do As I Can, Not As I Say: Grounding Language in Robotic Affordances.
CoRL 2022, PMLR 205:287–318. [arXiv:2204.01691](https://arxiv.org/abs/2204.01691)
- 9 Xie, J., Chen, Z., Zhang, R., et al. (2024).
Large Multimodal Agents: A Survey.
[arXiv:2402.15116](https://arxiv.org/abs/2402.15116)
- 10 Chen, M., Tworek, J., Jun, H., et al. (2021).
Evaluating Large Language Models Trained on Code (Codex, HumanEval, pass@k).
[arXiv:2107.03374](https://arxiv.org/abs/2107.03374)

- 11 Jimenez, C., Yang, J., Wettig, A., et al. (2024).
SWE-bench: Can Language Models Resolve Real-World GitHub Issues?
ICLR 2024. [arXiv:2310.06770](https://arxiv.org/abs/2310.06770)
- 12 Jain, N., Han, K., Gu, A., et al. (2024).
LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code.
[arXiv:2403.07974](https://arxiv.org/abs/2403.07974)
- 13 Ren, S., Guo, D., Lu, S., et al. (2020).
CodeBLEU: a Method for Automatic Evaluation of Code Synthesis.
[arXiv:2009.10297](https://arxiv.org/abs/2009.10297)
- 14 Zhou, S., Alon, U., Agarwal, S., Neubig, G. (2023).
CodeBERTScore: Evaluating Code Generation with Pretrained Models of Code.
EMNLP 2023. [arXiv:2302.05527](https://arxiv.org/abs/2302.05527)
- 15 Fried, D., Aghajanyan, A., Lin, J., et al. (2022).
InCoder: A Generative Model for Code Infilling and Synthesis.
ICLR 2023. [arXiv:2204.05999](https://arxiv.org/abs/2204.05999)

- 16 Zhou, S., Alon, U., Xu, F., Wang, Z., Jiang, Z., Neubig, G. (2022).
DocPrompting: Generating Code by Retrieving the Docs.
ICLR 2023. [arXiv:2207.05987](https://arxiv.org/abs/2207.05987)
- 17 Yang, J., Prabhakar, A., Narasimhan, K., Yao, S. (2023).
InterCode: Standardizing and Benchmarking Interactive Coding with Execution Feedback.
NeurIPS 2023. [arXiv:2306.14898](https://arxiv.org/abs/2306.14898)
- 18 Rozière, B., Gehring, J., Gloeckle, F., et al. (2023).
Code Llama: Open Foundation Models for Code.
[arXiv:2308.12950](https://arxiv.org/abs/2308.12950)
- 19 Lozhkov, A., Li, R., Allal, L. B., et al. (2024).
StarCoder 2 and The Stack v2: The Next Generation.
[arXiv:2402.19173](https://arxiv.org/abs/2402.19173)
- 20 Guo, D., Zhu, Q., Yang, D., et al. (2024).
DeepSeek-Coder: When the Large Language Model Meets Programming.
[arXiv:2401.14196](https://arxiv.org/abs/2401.14196)

- 21 Peng, S., Kalliamvakou, E., Cihon, P., Demirer, M. (2023).
The Impact of AI on Developer Productivity: Evidence from GitHub Copilot.
[arXiv:2302.06590](https://arxiv.org/abs/2302.06590)
- 22 Thakkar, P. (2023).
Copilot Internals (reverse engineering the Copilot prompt).
thakkarparth007.github.io/copilot-explorer